Python Dictionaries — Solutions & Explanations

Class XI Computer Science

Computer Science Department

October 21, 2026

Question 1: Sorting a Dictionary

Task: Write programs to sort the dictionary

```
{'ravi': '10', 'rajnish': '9', 'sanjeev': '15', 'yash': '2', 'suraj': '32'}
```

and print the sorted result.

Question 1: Solution

```python
d = {'ravi': '10', 'rajnish': '9', 'sanjeev': '15', 'yash': '2', 'suraj': '32'}

# 1. Sorted by keys (alphabetical order)
sorted_by_keys = dict(sorted(d.items()))
print("Sorted by Keys:", sorted_by_keys)

# 2. Sorted by values (numerical order)
sorted_by_values = dict(sorted(d.items(), key=lambda item: int(item[1])))
print("Sorted by Values:", sorted_by_values)
```

Key Takeaway

Dictionaries in modern Python (≥ 3.7) preserve insertion order. Passing `sorted(d.items())` to `dict()` yields an ordered dictionary.

Question 2: Highest 2 Values in a Dictionary

Task: Write a Python program to find the highest 2 values in a dictionary.

Question 2: Solution

```
marks = {'Physics': 88, 'Maths': 99, 'CS': 97, 'Chemistry': 92, 'English': 85}

# Extract unique values, sort descending, take top 2
unique_values = sorted(set(marks.values()), reverse=True)
top_2 = unique_values[:2]

print("Original Dictionary:", marks)
print("Highest 2 Values:", top_2)
```

Note

Using `set()` ensures duplicate highest values do not occupy both slots if unique highest tiers are desired.

Question 3: Character Frequency Dictionary

Task: Create a dictionary from a string that tracks the count of each letter.

Sample string: `'w3resource'`
Expected output: `{'3': 1, 's': 1, 'r': 2, 'u': 1, 'w': 1, 'c': 1, 'e': 2, 'o': 1}`

Question 3: Solution

```
text = 'w3resource'
freq = {}

for char in text:
    freq[char] = freq.get(char, 0) + 1

print("Character Frequencies:")
print(freq)
```

Explanation

The `dict.get(key, default)` idiom retrieves the existing tally if present; otherwise it returns 0, avoiding `KeyError`.

Question 4: Employee Salaries Input

Task: Write a program to enter names of employees and their salaries as input and store them in a dictionary. Here n is to be input by the user.

Question 4: Solution

```
n = int(input("Enter number of employees: "))
employees = {}

for i in range(n):
    name = input(f"Enter name of employee {i + 1}: ")
    salary = float(input(f"Enter salary for {name}: "))
    employees[name] = salary

print("\n--- Employee Records ---")
for name, salary in employees.items():
    print(f"{name}: Rs. {salary:.2f}")
```

Question 5: Count Character Appearances

Task: Write a program to count the number of times a character appears in a given string.

Question 5: Solution

```
s = input("Enter a string: ")
counts = {}

for ch in s:
    if ch in counts:
        counts[ch] += 1
    else:
        counts[ch] = 1

for char, count in counts.items():
    print(f"'{char}' : {count}")
```

Alternative

Can also be implemented concisely using Python's standard `collections.Counter(s)`.

Question 6: Number to Words Converter

Task: Convert a number entered by the user into its corresponding number in words.

Example: Input 876 $\rightarrow$ Output `'Eight Seven Six'`

Question 6: Solution

```
digit_map = {
    '0': 'Zero',  '1': 'One',   '2': 'Two',   '3': 'Three', '4': 'Four',
    '5': 'Five',  '6': 'Six',   '7': 'Seven', '8': 'Eight', '9': 'Nine'
}

num_str = input("Enter a number: ")
words = [digit_map[d] for d in num_str if d in digit_map]
result = " ".join(words)
print("Output:", result)
```

Question 7: Sports Team Win/Loss Records

Task: Repeatedly ask for a team name, wins, and losses. Store in a dictionary as `{team: [wins, losses]}`. Then:

- (a) Print a team's winning percentage
- (b) Create a list of the number of wins of each team
- (c) Create a list of teams with winning records

Question 7: Solution

```
teams = {}
n = int(input("How many teams? "))
for _ in range(n):
    name = input("Team name: ")
    w = int(input("Wins: "))
    l = int(input("Losses: "))
    teams[name] = [w, l]

# (a) Winning percentage
query = input("\nEnter team to query: ")
if query in teams:
    w, l = teams[query]
    total = w + l
    pct = (w / total * 100) if total > 0 else 0.0
    print(f"{query} Winning Percentage: {pct:.2f}%")

# (b) List of win counts
wins_list = [record[0] for record in teams.values()]
print("Wins across all teams:", wins_list)

# (c) Teams with winning records
winning_teams = [t for t, (w, l) in teams.items() if w > l]
print("Teams with winning records:", winning_teams)
```

Question 8: Product Price Lookup

Task: Repeatedly ask the user to enter product names and prices, storing them in a dictionary. Then allow repeated lookups of product prices (or a message if not found).

Question 8: Solution

```
catalog = {}
print("Enter product data (type 'done' to stop):")
while True:
    prod = input("Product name: ").strip()
    if prod.lower() == 'done':
        break
    price = float(input(f"Price for {prod}: "))
    catalog[prod] = price

print("\n--- Product Search (type 'exit' to quit) ---")
while True:
    query = input("Search product: ").strip()
    if query.lower() == 'exit':
        break
    if query in catalog:
        print(f"Price of {query}: Rs. {catalog[query]:.2f}")
    else:
        print(f"'{query}' is not available in the catalog.")
```

Question 9: Month Days & Analysis

Task: Create a dictionary of month names and number of days. Then:

- (a) Ask for a month and tell how many days it has
- (b) Print all keys in alphabetical order
- (c) Print all months with 31 days
- (d) Print key-value pairs sorted by number of days

Question 9: Solution

```
months = {
    'January': 31, 'February': 28, 'March': 31, 'April': 30,
    'May': 31, 'June': 30, 'July': 31, 'August': 31,
    'September': 30, 'October': 31, 'November': 30, 'December': 31
}

# (a) Query days
m = input("Enter month name: ").capitalize()
print(f"Days in {m}:", months.get(m, "Month not found"))

# (b) Keys in alphabetical order
print("Alphabetical:", sorted(months.keys()))

# (c) Months with exactly 31 days
print("31-day months:", [m for m, d in months.items() if d == 31])

# (d) Sorted by days
print("Sorted by days:", sorted(months.items(), key=lambda x: x[1]))
```

Question 10: Multi-Field Student Records

Question: Can you store the details of 10 students in a dictionary at the same time? Details include rollno, name, marks, grade etc. Give an example to support your answer.

Question 10: Solution

Answer: Yes. Use the roll number as the unique key and a nested dictionary (or list/tuple) as the value.

```
students = {
    101: {'name': 'Aarav', 'marks': 94.5, 'grade': 'A1'},
    102: {'name': 'Bhavna', 'marks': 88.0, 'grade': 'A2'},
    103: {'name': 'Chirag', 'marks': 76.5, 'grade': 'B1'},
    # ... up to 10 students
    110: {'name': 'Jaya', 'marks': 91.0, 'grade': 'A1'}
}

print("Roll 101 Name:", students[101]['name'])
print("Roll 101 Grade:", students[101]['grade'])
```

Question 11: Inverting a Dictionary

Task: Given `x = {'k1':'v1', 'k2':'v2', 'k3':'v3'}`, create the inverted dictionary:

```
inverted_x = {'v1': 'k1', 'v2': 'k2', 'v3': 'k3'}
```

Question 11: Solution

```
x = {'k1': 'v1', 'k2': 'v2', 'k3': 'v3'}

# Dictionary comprehension
inverted_x = {value: key for key, value in x.items()}

print("Original:", x)
print("Inverted:", inverted_x)
```

Constraint

This approach assumes that all values in the original dictionary are unique and hashable (strings, ints, tuples, etc.).

Question 12: Overlapping Keys

Task: Given two dictionaries D1 and D2, write a program that lists the overlapping keys (keys that appear in both dictionaries).

Question 12: Solution

```
D1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
D2 = {'b': 20, 'd': 40, 'e': 50}

# Method 1: Set intersection (idiomatic)
common_keys = list(D1.keys() & D2.keys())
print("Overlapping keys (Set):", common_keys)

# Method 2: List comprehension
overlapping = [k for k in D1 if k in D2]
print("Overlapping keys (List):", overlapping)
```

Question 13: Detecting Duplicate Values

Task: Check if two same values in a dictionary have different keys.

For `D1 = {'a': 10, 'b': 20, 'c': 10}` print “2 keys have same values”.
For `D2 = {'a': 10, 'b': 20, 'c': 30}` print “No keys have same values”.

Question 13: Solution

```
def check_duplicate_values(d):
    vals = list(d.values())
    if len(vals) != len(set(vals)):
        print(f"For {d}: 2 keys have same values")
    else:
        print(f"For {d}: No keys have same values")

D1 = {'a': 10, 'b': 20, 'c': 10}
D2 = {'a': 10, 'b': 20, 'c': 30}

check_duplicate_values(D1)
check_duplicate_values(D2)
```

Logic

If `len(values) > len(set(values))`, at least one value appears more than once across different keys.

Question 14: Sub-Dictionary Verification

Task: Check if a dictionary is contained in another dictionary.

Example:
`d1 = {1:11, 2:12}`
`d2 = {1:11, 2:12, 3:13, 4:15}`
Then `d1` is contained in `d2`.

Question 14: Solution

```
d1 = {1: 11, 2: 12}
d2 = {1: 11, 2: 12, 3: 13, 4: 15}

# Method 1: Set comparison on items
is_contained = d1.items() <= d2.items()
print("d1 is contained in d2:", is_contained)

# Method 2: Explicit verification
contained = all(k in d2 and d2[k] == v for k, v in d1.items())
print("Verified via generator:", contained)
```

Question 15: Summing List Elements as Values

Task: Dictionary D1 has values as lists of numbers. Create D2 with the same keys but values as the sum of the list elements.

Example: `D1 = {'A': [1, 2, 3], 'B': [4, 5, 6]}`
Then `D2 = {'A': 6, 'B': 15}`

Question 15: Solution

```
D1 = {'A': [1, 2, 3], 'B': [4, 5, 6]}

# Dictionary comprehension with sum()
D2 = {key: sum(values) for key, values in D1.items()}

print("Original D1:", D1)
print("Computed D2:", D2)
```

Question 16: Phonebook Operations

Task: Input friends' names and phone numbers into a dictionary, then perform:

- (a) Display all names and numbers
- (b) Add a new key-value pair
- (c) Delete a particular friend
- (d) Modify the phone number of an existing friend
- (e) Check if a friend is present
- (f) Display the dictionary sorted by name

Question 16: Solution

```
phonebook = {'Ananya': '9847012345', 'Rohan': '9447123456'}

# (a) Display all
print("(a) All contacts:", phonebook)

# (b) Add new contact
phonebook['Devika'] = '9895011223'
print("(b) Added Devika:", phonebook)

# (c) Delete a friend
phonebook.pop('Rohan', None)
print("(c) Deleted Rohan:", phonebook)

# (d) Modify phone number
if 'Ananya' in phonebook:
    phonebook['Ananya'] = '9744000000'
print("(d) Modified Ananya:", phonebook)

# (e) Check membership
print("(e) Is Ananya present?", 'Ananya' in phonebook)

# (f) Sorted by name
print("(f) Sorted contacts:", dict(sorted(phonebook.items())))
```